

Q) Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP.

Use a Tree or an undirected graph for BFS and DFS.

BFS-

```
#include <iostream>    // For input and output
#include <stdlib.h>     // For standard library functions
#include <queue>        // For queue data structure used in
BFS
#include <omp.h>        // For OpenMP parallel programming
```

```
using namespace std;
```

```
// Node class for binary tree
```

```
class node
```

```
{
```

```
public:
```

```
    node *left, *right; // Left and right child pointers
```

```
    int data;           // Data stored in node
```

```
};
```

```
// Function to insert node in binary tree level-wise
```

```
node *insert(node *root, int data)
```

```
{
```

```
    // If tree is empty, create root node
```

```
    if (!root)
```

```
    {
```

```
        root = new node;    // Create new node
```

```
        root->left = NULL;  // Initially left child is NULL
```

```
        root->right = NULL; // Initially right child is NULL
```

```
        root->data = data;  // Store data
```

```
    return root;          // Return root
}
```

```
queue<node *> q;           // Queue for level-wise insertion
q.push(root);             // Start from root node
```

```
// Continue until correct empty position is found
while (!q.empty())
```

```
{
    node *temp = q.front(); // Get front node
    q.pop();               // Remove it from queue
```

```
// If left child is empty, insert new node there
if (temp->left == NULL)
```

```
{
    temp->left = new node;
    temp->left->left = NULL;
    temp->left->right = NULL;
    temp->left->data = data;
    return root;
}
```

```
else
```

```
{
    q.push(temp->left); // If left child exists, push it into
```

```
queue
}
```

```
// If right child is empty, insert new node there
if (temp->right == NULL)
```

```
{
    temp->right = new node;
    temp->right->left = NULL;
    temp->right->right = NULL;
```

```

        temp->right->data = data;
        return root;
    }
    else
    {
        q.push(temp->right); // If right child exists, push it
into queue
    }
}

return root;          // Return root
}

// Function to perform BFS traversal
void bfs(node *head)
{
    queue<node *> q;      // Queue used for BFS
    q.push(head);        // Start BFS from root node

    int qSize;           // Stores number of nodes at current
level

    // Continue until queue becomes empty
    while (!q.empty())
    {
        qSize = q.size(); // Number of nodes present at
current level

        // Process current level nodes in parallel
        #pragma omp parallel for
        for (int i = 0; i < qSize; i++)
        {
            node *currNode; // Current node to be processed

```

```

        // Critical section because queue is shared by all
threads
        #pragma omp critical
        {
            currNode = q.front();    // Take node from front of
queue
            q.pop();                // Remove node from queue
            cout << "\t" << currNode->data; // Print current
node
        }

```

```

        // Critical section because multiple threads may push
children
        #pragma omp critical
        {
            // If left child exists, add it to queue
            if (currNode->left)
                q.push(currNode->left);

            // If right child exists, add it to queue
            if (currNode->right)
                q.push(currNode->right);
        }
    }
}

```

```

// Main function
int main()
{
    node *root = NULL;    // Initially tree is empty
    int data;             // Data to insert

```

```

    char ans;                // User choice for continuing
insertion

    // Take input from user and insert nodes
do
{
    cout << "\nEnter data => ";
    cin >> data;            // Read data

    root = insert(root, data); // Insert data into tree

    cout << "Do you want to insert one more node? ";
    cin >> ans;            // Read user choice

} while (ans == 'y' || ans == 'Y');

cout << "\nBFS Traversal:\n";

bfs(root);                // Call BFS function

return 0;                // End program
}

```

Output-

Enter data => 2
Do you want to insert one more node? y

Enter data => 4
Do you want to insert one more node? y

Enter data => 3

Do you want to insert one more node? y

Enter data => 5

Do you want to insert one more node? y

Enter data => 7

Do you want to insert one more node? y

Enter data => 8

Do you want to insert one more node? y

Enter data => 9

Do you want to insert one more node? y

Enter data => 10

Do you want to insert one more node? n

BFS Traversal:

2 4 3 5 7 8 9 10

...Program finished with exit code 0

Press ENTER to exit console.

DFS-

```
#include <iostream>    // For input and output
#include <vector>       // For adjacency list representation
#include <stack>        // For stack used in DFS
#include <omp.h>        // For OpenMP parallel programming
```

```
using namespace std;
```

```
const int MAX = 100000;    // Maximum number of nodes
```

```
vector<int> graph[MAX];    // Adjacency list for graph
bool visited[MAX];        // Visited array to track visited
nodes
```

```
// Function to perform DFS traversal
```

```
void dfs(int node)
```

```
{
```

```
    stack<int> s;          // Stack used for DFS
    s.push(node);          // Push starting node into stack
```

```
    // Continue until stack becomes empty
```

```
    while (!s.empty())
```

```
    {
```

```
        int curr_node = s.top(); // Get top node from stack
        s.pop();                // Remove it from stack
```

```
        // If current node is not visited
```

```
        if (!visited[curr_node])
```

```
        {
```

```
            visited[curr_node] = true; // Mark current node as
visited
```

```
cout << curr_node << " "; // Print current node
```

```
// Check all adjacent nodes in parallel
```

```
#pragma omp parallel for
```

```
for (int i = 0; i < graph[curr_node].size(); i++)
```

{

```
int adj_node = graph[curr_node][i]; // Get adjacent
```

node

```
// If adjacent node is not visited
```

```
if (!visited[adj_node])
```

{

```
// Critical section because stack is shared
```

```
#pragma omp critical
```

{

```
s.push(adj_node); // Push adjacent node into
```

stack

}

}

}

}

}

}

```
// Main function
```

```
int main()
```

{

```
int n, m, start_node;    // n = nodes, m = edges,
```

```
start_node = starting point
```

```
cout << "Enter number of nodes, number of edges and  
starting node:\n";
```

```
cin >> n >> m >> start node:
```



```

cout << "Enter pairs of connected nodes:\n";

// Input graph edges
for (int i = 0; i < m; i++)
{
    int u, v;          // Edge between u and v
    cin >> u >> v;

    graph[u].push_back(v); // Add v to u's adjacency list
    graph[v].push_back(u); // Add u to v's adjacency list
    because graph is undirected
}

// Initialize all nodes as not visited
#pragma omp parallel for
for (int i = 0; i < n; i++)
{
    visited[i] = false;
}

cout << "\nDFS Traversal:\n";

dfs(start_node);      // Start DFS from given node

return 0;             // End program
}

```

Output-

Enter number of nodes, number of edges and starting node:

4

4

1

Enter pairs of connected nodes:

1 2

1 3

1 4

3 2

DFS Traversal:

1 4 3 2

...Program finished with exit code 0

Press ENTER to exit console.